

Beginner AI Agent Tutorial

1. What is an AI agent?

An AI agent is a system that can:

- receive input,
- reason about it,
- use tools or memory,
- and respond with an action or answer.

In this project, the agent will:

- take a user question,
- fetch relevant context using RAG,
- optionally use Ollama to generate a response,
- and answer in a simple web interface.

2. What are the main ideas?

2.1 RAG

RAG stands for Retrieval-Augmented Generation.

Instead of relying only on the model's memory, the system first retrieves relevant information from a knowledge base and then sends that information to the model.

2.2 Vector database

A vector database stores embeddings, which are numerical versions of text. Similar text gets similar vectors, so the system can search by meaning rather than only exact words.

2.3 Ollama

Ollama runs local LLMs on your machine. This lets you test open-source models without paying for remote APIs.

2.4 Memory

Memory lets the agent remember previous conversation turns. In this project, memory is kept lightweight so the agent does not get confused by unrelated old chat.

3. Project architecture

This project includes:

- Python backend
- local LLM via Ollama
- vector database using FAISS
- knowledge file for RAG context
- FastAPI API
- React frontend

The flow is:

1. User asks a question.
2. The agent searches the knowledge base.
3. Relevant chunks are retrieved.
4. The retrieved content is sent as context.
5. The model generates an answer.
6. The result is returned to the frontend.

4. Step-by-step setup

Step 1: Create a project folder

Create a folder named ``my-first-ai-agent``.

Step 2: Create a virtual environment

Run:

```
python3 -m venv .venv  
source .venv/bin/activate
```

Step 3: Install dependencies

Install the Python packages:

```
pip install sentence-transformers faiss-cpu numpy requests fastapi uvicorn
```

Step 4: Install Ollama

Download and install Ollama from the official website.

Then run:

```
ollama pull llama3.2
```

Step 5: Create the knowledge file

Add a file like `knowledge.txt` containing your domain knowledge. For example:

AI Agent

An AI agent is a software system that can take input, reason about it, and choose an action.

RAG

RAG stands for Retrieval-Augmented Generation.

Step 6: Build the vector store

Use a sentence embedding model so every chunk is converted into a vector. Then store the vectors in FAISS for semantic search.

Step 7: Add the agent logic

The agent should:

- receive a question,
- search for relevant documents,
- build a prompt with the retrieved context,
- call Ollama,
- return the answer.

5. The agent loop

The basic agent logic is:

1. read user input,
2. search the vector DB,
3. build a prompt,
4. send it to the model,
5. return the final answer,
6. optionally store short-term memory.

This is the core pattern behind many RAG-based agents.

6. Why RAG matters

Without RAG, the model only answers from what it learned during training.

With RAG, the model uses your actual knowledge base as an external source.

This is useful when:

- you need up-to-date information,
- you want custom company knowledge,
- you want answers grounded in internal docs.

7. Running the project locally

Start Ollama

```
ollama serve
```

Run the Python app

```
python main.py
```

Run the API

```
cd backend
```

```
uvicorn app:app --host 0.0.0.0 --port 8000
```

Run the frontend

```
cd frontend
```

```
npm install
```

```
npm run dev -- --host 0.0.0.0
```

8. Important beginner lessons

Lesson 1: context is separate from memory

Always keep:

- retrieval context from the knowledge base,
- conversational memory separate.

This avoids mixing unrelated history into a new question.

Lesson 2: keep prompts clear

A good prompt says:

- use retrieved context first,
- use reasoning second,
- be honest about missing facts,
- only fall back when the model is unavailable.

Lesson 3: math questions need special handling

If the user asks a simple arithmetic question, the model should answer directly instead of pulling in unrelated conversation history.

9. Real-world improvements

Once you understand the basics, you can upgrade the project with:

- Qdrant or Weaviate instead of FAISS,
- PDF ingestion for internal documents,
- tool calling,
- Redis or database memory,
- authentication,
- deployment to Render or Hugging Face Spaces.

10. Final summary

This project teaches the building blocks of modern AI agents:

- local model execution with Ollama,
- retrieval with vector search,
- grounded answers via RAG,
- memory and context management,
- a simple API and interface.

You now have a working understanding of how these pieces fit together.

11. Suggested next steps

1. Add more knowledge to `knowledge.txt`.
2. Test different models in Ollama.
3. Add PDF ingestion.
4. Upgrade to Qdrant.
5. Add tool calling and long-term memory.

12. Quick recap

A simple AI agent is:

- input -> retrieval -> reasoning -> response.

RAG helps by grounding responses in facts you control.

Vector DB helps by finding semantically similar information quickly.

Ollama lets you run models locally for learning and experiments.

This is the foundation of many real AI products.